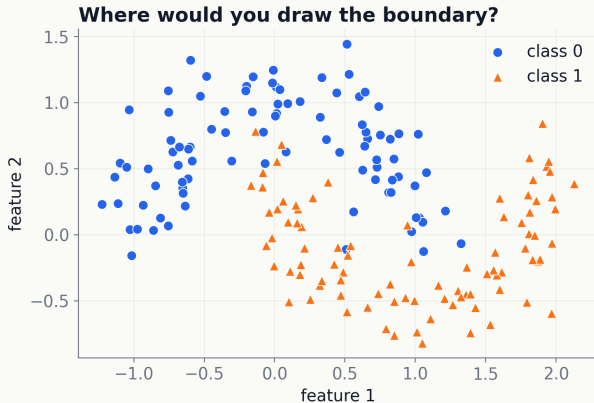


Can several neurons learn this curved boundary?

We will build the pieces, trace one learning step, then train a complete model.

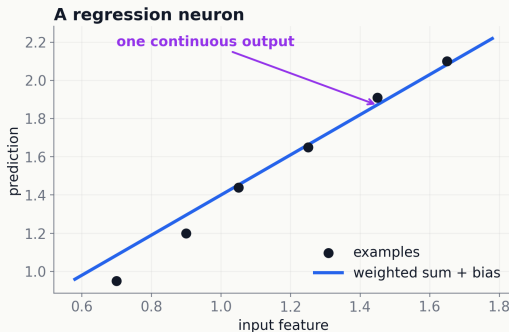
Main question: what travels forward, and what travels backward?



Last week's linear model was already one neuron

$$\hat{y} = x^T w + b$$

- ▶ inputs: the measured features;
- ▶ parameters: weights and bias;
- ▶ output: one continuous prediction.



A neuron adds an activation after a weighted sum

First calculate a score:

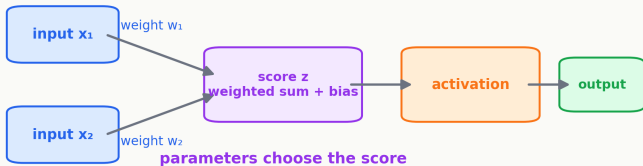
$$z = x^T w + b.$$

Then apply a function:

$$a = \text{activation}(z)$$

The activation controls what the neuron sends forward.

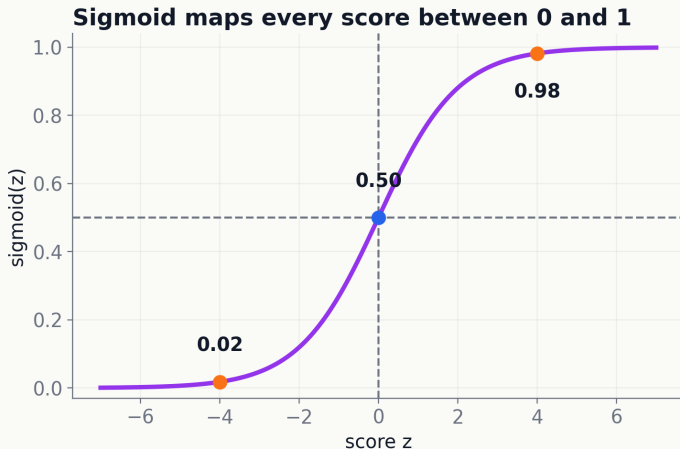
Live: change an input, weight, and bias



Sigmoid turns a score into a probability

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- ▶ large negative score: near 0;
- ▶ zero score: exactly 0.5;
- ▶ large positive score: near 1.



One neuron still draws one straight boundary

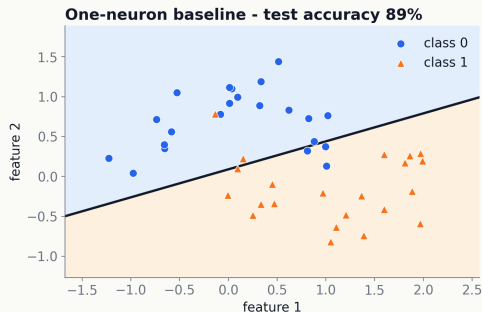
Predict class 1 when

$$\sigma(x^\top w + b) \geq 0.5.$$

The class changes at

$$x^\top w + b = 0,$$

one straight line in two dimensions.



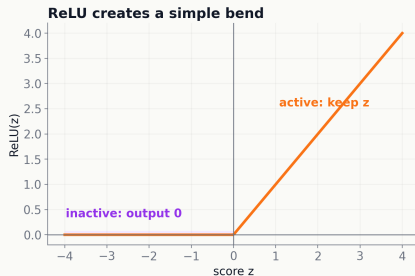
ReLU helps hidden neurons create new features

$$\text{ReLU}(z) = \max(0, z)$$

negative score $\rightarrow 0$; positive score $\rightarrow z$.

This small bend lets the network learn nonlinear boundaries.

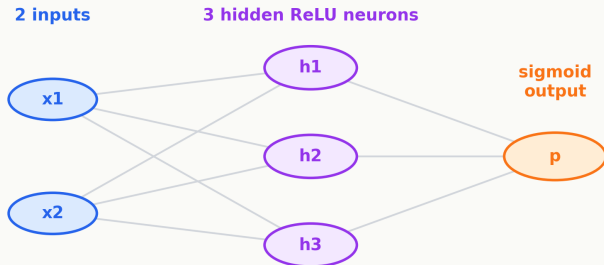
Live: compare Linear and ReLU



An MLP repeats the same small calculation

- ▶ each arrow has a weight;
- ▶ each neuron has a bias;
- ▶ hidden neurons use ReLU;
- ▶ the output uses sigmoid.

Hidden activations are new features learned by the network.



every arrow is an adjustable weight

A forward pass calculates from left to right

$$z_1 = XW_1 + b_1$$

$$h = \text{ReLU}(z_1)$$

$$z_2 = hW_2 + b_2$$

$$p = \sigma(z_2)$$

Keep the intermediate values. Backpropagation will need them.

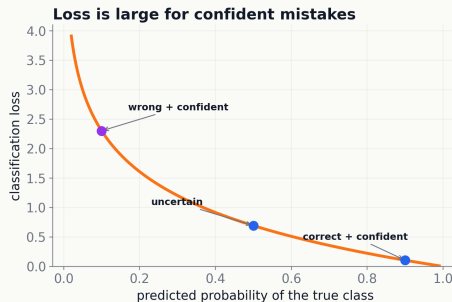
Notebook checkpoint 6 Predict each array's shape before running the forward-pass cell.

Classification loss measures confident mistakes

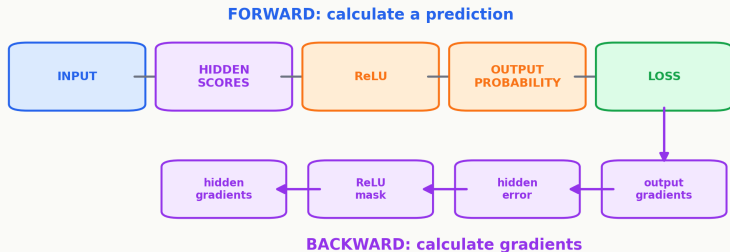
For a true class of 1:

- ▶ probability 0.9: small loss;
- ▶ probability 0.5: uncertain;
- ▶ probability 0.1: large loss.

The loss creates the error signal used for learning.



Backpropagation sends the error signal backward



Each operation uses a local rule, then passes the error signal backward.

Optional interactive: trace the two directions

Each parameter receives a gradient and an update

$$\begin{aligned}\delta_2 &= (p - y)/n, \\ \nabla W_2 &= h^\top \delta_2, \\ \delta_1 &= (\delta_2 W_2^\top) \odot [z_1 > 0], \\ \nabla W_1 &= X^\top \delta_1.\end{aligned}$$

- ▶ output error first;
- ▶ output-layer gradients;
- ▶ hidden error through ReLU;
- ▶ hidden-layer gradients.

new parameter = old parameter $- \eta \times$ gradient

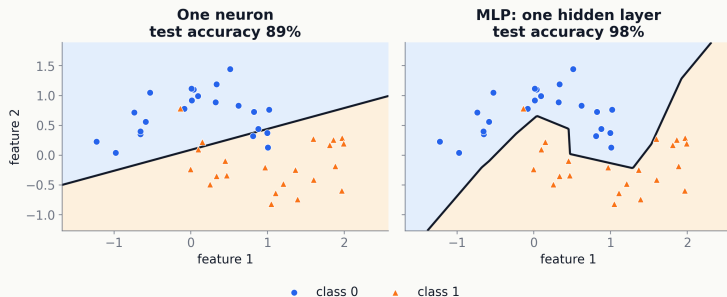
The goal is to read the direction and shapes, not memorize these formulas.

scikit-learn trains the same architecture for us

```
model = make_pipeline(  
    StandardScaler(),  
    MLPClassifier(  
        hidden_layer_sizes=(8,),  
        random_state=7  
    )  
)  
  
model.fit(X_train, y_train)  
predictions = model.predict(X_test)
```

- ▶ fit: forward, loss, backward, update, repeat;
- ▶ predict: forward pass only;
- ▶ the same learning loop is inside the library.

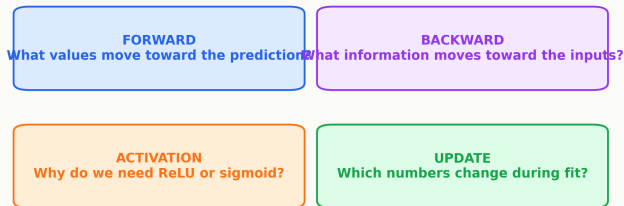
The hidden layer bends the decision boundary



Compare the shape and the mistakes - not only the accuracy number.

Live: build the boundary in TensorFlow Playground

Explain both directions



Explain the process without saying “the computer just learns.”

Complete the story using **parameters**, **activation**, **loss**, **gradient**, and **update**.